

CHAPTER

3

Access Control

Access control, or authorization, is the process of limiting users to access only the functionality and data that they are specifically permitted to use. The Principle of Least Privilege, or POLP, defines the essence of what a good access control mechanism should deliver. As Jerome Saltzer stated in 1974, “Every program and every privileged user of the system should operate using the least amount of privilege necessary to complete the job.”

The Principle of Least Privilege applies not just to access control *within* your application, but as we discuss elsewhere in this book, it should also be applied to every other facet of your application architecture as part of a strong defense-in-depth strategy.

Even in a simple web application, access control appears at many layers: within the application itself, as part of the web server software, the J2EE container, the operating system the web server is installed on, the database, and several other layers that each have their own different flavor of access control. This chapter focuses on application-specific access control that you as a developer need to build within your Java web applications. We are not describing the access control mechanism needed for an operating system, a router, or other network appliance. Each of these requires a form of access control that is quite different in implementation and design compared to a web application or web service. We will discuss the various theories behind access control design, but ultimately our goal in this chapter is to provide you with a path toward building a modern access control system that is appropriate for your application.

Unlike other risks, such as SQL injection or clickjacking, there is no simple answer to the question “What kind of access control should I design for my application?” This is actually a problem for the developer. When there are many different schemes for implementing access control, and seemingly every framework has its own implementation, making the right choice is often difficult. Worse, sometimes developers just use the built-in access control mechanism that comes with their application framework, only to find out mid-project that the built-in access control APIs are inadequate for their needs. Or even worse, some developers do not even think about access control until late in the project, if at all. Some may say, Damn the torpedoes, make everyone an admin! But do not give in to the dark side. Because access control is so design heavy and becomes integral to the security nervous system in your software, it is important to carefully consider access control from the very earliest stages of your project.

Overall, the simpler your software, the more viable built-in framework-based access control patterns will work for you. However, the opposite is also true. The more complex your software, the more likely you will need to build some or all of your access control mechanism as custom code specific to your application or business.

Identity and Access Control

Before we go any further, let’s clarify some terms. *Authorization* and *authentication* are two separate but related parts of your security paradigm. *Authentication* is the process of verifying a user’s identity. *Authorization*, or access control, is the process of determining what resources an authenticated user is able to access. For access control to function properly, you must first

have a securely authenticated user, as shown in [Figure 3-1](#). For more information about authentication, see [Chapter 2](#).

Image

FIGURE 3-1. *Access control workflow*

An important first step is to define the core elements of what makes up an access control mechanism. Access control is the security layer in software that limits users to accessing only authorized functionality and data. Let's break down this definition. Access control is.

Image a layer of defense: Access control is a core control in all software.

Image that limits: By default users get no access.

Image users: Users must already be authenticated.

Image to accessing only authorized functionality and data: Users are defined by roles or are given a list of entitlements or capabilities that allow them to access certain data or functionality.

Let's dig deeper into the nomenclature of access control and define some of the core terms shown in [Figure 3-2](#):

Image

FIGURE 3-2. *Attribute-based access control (ABAC)*

Image Subject The is the end user or service/application making the access request.

Image Subject attributes These are attributes that define the user or entity making the request. This includes authentication status, the group or groups the user is in, the role the user or user group is in, and other information about the entity making the request.

Image Group A group is a basic organizational structure. This can be a group of users, a group of activities, or another relevant access control group (IT support group; marketing group; "Edit Car" group of permissions, including view car, edit car, update car).

Image Role A functional abstraction to uniquely describe system collaborators with similar or unique duties. While specific roles are unique to individual systems, some roles are found in many systems—for example, content editor, content creator, content publisher, system administrator, and so on. *Roles* are different from *groups* in that roles ascribe functions that in many cases may be performed by many individuals across different groups or areas of an organization. Roles may also be inherited from other roles. For example, if a user has the ability to modify an object, that user must also have implicit read access.

Image Access control rules, or entitlements An access control rule (or entitlement) is a logical decision or series of decisions that need to be made in order to determine if a subject is allowed to access some activity, feature, or data. Depending on the type of access control, these rules can define access to certain features or objects or combinations of both.

Image **Access control policy** An access control policy is the total collection of access control rules that make up your software.

Image **Policy Information Point (PIP)** This is a general term that describes any attribute source potentially needed to make an access control decision (subject/user data; object data; environmental data, such as time or location; and so on).

Image **Action** An action is some kind of system feature, such as read article, modify article, create new article, execute batch process, schedule backup, or edit user.

Image **Policy Enforcement Point (PEP)** This is the area in application code where the access control check is made. This code needs to properly handle both an accepted and a rejected decision.

Image **Policy Decision Point (PDP)** This is the access control engine that takes all the attribute inputs and applicable access control policy for a specific request, and evaluates them to make access control decisions. While a PEP is an area of code where an access control check is made, the PDP is the engine that does the work behind the scenes of an access control check.

Image **Policy Administration Point (PAP)** This is the administrative entry into the access control system that lets privileged staff create new policy and apply it to subjects and groups.

Image **Object** An object is the data being operated on. This could be a document, schedule, or company (different permissions are applicable to different types of object resources).

Image **Object attributes** These are attributes that define the type of object being operated on. This could include any classification such as data sensitivity level, parent-child data relationships, or other data category.

Let's™ put it all together. The following is a pseudocode example of a Java EE access control filter:

Image

Each access control design type will involve different interactions between these core components. In certain types of access control, single components will merge multiple concepts defined earlier. For example, in role-based access control (RBAC), the PIP, PEP, and PDP are essentially all rolled into one. In an attribute-based access control (ABAC) system, these are all separate and discrete software components and services.

Before we discuss positive mechanisms for access control, let's™ talk about ways an attacker can abuse vulnerabilities in access control designs or implementations.

Attacks on Access Control

The most common form of attack against a web application's access control mechanism is called *privilege escalation*. This is when a (normally authenticated) attacker is able to manipulate input in a way that allows him to access features or data he should not be able to access. For example, in Vertical Privilege Escalation an attacker may register for a standard user account and, via malicious input manipulation or other techniques, may access administration functionality. Often this takes the form of simply browsing to a known (or guessed) administration URL. Tools exist that can rapidly cycle through long lists of known or random URL paths, looking for

responses that differ from the standard “404 Not Found” page. This is why “security by obscurity” is not a valid access control strategy.

Another attack on access control is a Horizontal Privilege Escalation attack. In Horizontal Privilege Escalation, two users may have the same level of permissions but an attacker may leverage one user account to maliciously access another user’s private data. For example, this could occur by manipulating the from and to account numbers in a bank transfer request:

Image

Other attacks on access control include the abuse of business logic or abuse of workflow. For example, a normal workflow at an eCommerce website would include steps such as adding an item to a cart, setting the shipping address for delivery, setting the payment address, entering payment information, successfully processing the payment, and then completing the checkout. Attackers may attempt to abuse this workflow by skipping the payment steps.

Access Control Anti-Patterns and Design Flaws

There are many ways that an access control mechanism can go south.

Anti-Pattern 1: Avoiding Access Control Features

The first design flaw of access control design is *not using it*. On several occasions we have reviewed an application for which all users had complete access. When asked about the access control mechanism, the answer in this situation is often “we’ll slap in on later.” FAIL. Access control is central to your application’s defense-in-depth strategy and the implementation needs to be carefully considered throughout the entire software development lifecycle.

Anti-Pattern 2: Using Request Data to Make Access Control Policy Decisions

Another common design flaw is access control that depends solely on client-side data to determine a user’s role or entitlement. It is trivial for an attacker to modify request data using an intercepting proxy or similar tool. Attackers can and will modify any aspect of the HTTP Request so that a cookie with `ROLE=USER` can easily be modified to be `ROLE=ADMIN`. Or how about this snippet of a form where the user’s role is driven by a hidden variable?

Image

In this case, changing the role parameter to `admin` will defeat the application’s access control and allow the comment to bypass the standard comment moderation process. Not only is this a failure of access control, but this is also a failure of authentication and session management because the user ID is driven from the request. EPIC FAIL. We must never make access control decisions where the user’s entitlements or permissions are based solely on data from the client such as cookies, hidden form fields, URL parameters, or anything else that comes from the request.

On this same line of thinking, you must never depend on access control decisions that are made solely in JavaScript, such as whether or not to render administrator-only links depending on the value of some variable or cookie. It is safe to assume that all client-side code will be inspected, and any aspect of the request can and will be modified by the attacker. Access control decisions *must* come from *server-side* access control policy data.

Anti-Pattern 3: Relying on Obscurity for Access Control

Another design flaw that we have confronted in our careers is relying on obscurity in the access control design of an application. (Actually, we worked on a large application that depended on this “security.”) For example, what do you think of the following URL protecting a sensitive file where no other access control mechanism is in place?

~~~~~ [www.mydocuments.com/download/9982e094925d19aa1b122da5f1dbcd86/the\\_master\\_plan.ppt](http://www.mydocuments.com/download/9982e094925d19aa1b122da5f1dbcd86/the_master_plan.ppt)

Although this may look hard to guess, this link is depending on “security by obscurity,” which is almost always an epic fail. The URL can be “stolen” via shoulder surfing while someone is browsing; it will potentially leak in browser history; it can be bookmarked and emailed around; and it will appear in proxy and web server logs. Obscurity does not equal access control and “hidden” URLs become visible over time. You need something better.

### Anti-Pattern 4: Hard Coded Policy

*“authorization policies are often hard coded into application code making it difficult and expensive to adapt to changes in the business environment. Fine-grained authorization and entitlements need to be managed differently than the traditional entitlements. The authorization requirements should be given enough emphasis right from the initial phases of software development process. ([www.infoq.com/news/2010/10/javaone2010-fga](http://www.infoq.com/news/2010/10/javaone2010-fga))*

One of the major components of any access control system is the Policy Enforcement Point, or PEP. This is the area in application code where an access control check is added. The following example illustrates one of the most common anti-patterns that you see in many Java applications: custom access control checks made in code at the Policy Decision Point.

Image

The critical design flaw here, as well as one of the most prominent features in many software frameworks, is the merging of access control policy and application code. This “hard coding” of access control policy leads to a variety of challenges.

When you “hard code” policy in your application code like this, you need to push a new version of your software any time your access control policy changes. It also makes it very difficult to ensure that access control is consistent across every place that a similar check has to be made. Maybe you changed the policy at the check that decides whether to display the “Delete Company” button, but did you also remember to change the policy on the actual action that deletes that company from the database?

This anti-pattern makes often essential tasks such as co-branding and customization nearly impossible, or at least expensive, as multiple branches or forks of your application need to be maintained simultaneously.

When auditing web application software for security, you often find hundreds, if not thousands, of hard-coded role checks in application code at all layers of software. When security professionals are attempting to audit the access control policy of complex software, they often have to pore through millions of lines of code if that policy is hard-coded. This makes it very difficult to prove that the software has adequate authorization controls in place. This also makes the code less safe because it will be more difficult to match the supposed access control policy to the reality of how the software operates. This is a problem for any complex access control system where policy is hard-coded and is not just a critique on role-based access control.

## Anti-Pattern 5: Adding Access Control Manually to Every Endpoint

Another common access control anti-pattern is an application that requires access control logic to be manually added to every endpoint in your application. This is similar to the problem of hard-coded policy in that access control policy needs to be manually added to the application in some way.

Another problem with hard coding your policy is the challenge of auditing the security of your application. When a security auditor reviews your code for security and asks about your access control mechanism, is your answer, “Yeah, my policy is in the code, just go read those 7 million LOC and there is your policy.” If so, FAIL. No one likes auditing complex access control policy sprinkled across thousands of lines of code except for hourly consultants’ just don’t rush them.

## Anti-Pattern 6: Fail Open

Yet another common problem with access control mechanisms is the “fail open” flaw. When reviewing a web application’s access control, a regular quality analysis (Q/A) process might verify that standard web application user behavior limits access properly. However, certain attack scenarios outside of the application’s normal operation may cause program errors that cause the access control mechanism to fail. For example, consider the processing of a request parameter in Java. In normal use, `request.getParameter()` will return either an empty string or the data that was submitted. But what if the attacker uses an intercepting proxy and fully removes the parameter from the request? The `getParameter()` method then returns `NULL`, something the programmer may not have prepared for. The resulting `NullPointerException` may cause a failure that grants access. This may seem unlikely but does indeed happen on occasion.

**Fail Open Access Control Example** When processing form input from the request, there are three basic possibilities:

Image The parameter could have been left empty, which results in an empty string.

Image The parameter may have data entered, which would return a string containing the submitted data.

Image An attacker may use an intercepting proxy to remove the entire parameter, which results in the value being null. This could cause a `NullPointerException`, which, if not handled correctly, could circumvent some access control checks. Because `NullPointerException` is a subclass of `RuntimeException`, it will not be flagged as a potential issue by the compiler. Thus, developers might not even consider this possibility until it is too late.

Imagine the following code where the developer never considered the possibility that the debug parameter might be null. Q/A would have missed this as well if they were only testing the “normal” operation of the site!

Image

## Positive Access Control Patterns

Here are several guiding principles to help you build a robust and secure access control mechanism.

## Positive Pattern 1: Consider a Centralized Enforcement Layer

One of the key mechanisms of an access control system is where you are going to apply the access control checks in code—the Policy Enforcement Point, or PEP. There are many ways to implement this, but ideally it is done with a centralized component that all requests are forced to go through, as opposed to checks that developers need to manually add to code at various places in the application.

Java has an excellent mechanism to force all requests to go through a centralized enforcement layer, the Java EE request filter. Filters provide the capability to intercept and modify, redirect, or block requests and responses. They can be configured to apply to certain URL paths in the web application deployment descriptor (web.xml) so access control policy changes need not require changes to application code. For example, a configuration that allows only admin-level users to access the admin portions of a site might look like this:

Image

An application taking full advantage of this paradigm might have a URL structure as follows:

Image /admin Only accessible by superuser-level users.

Image /company/123 Accessible by anyone with permission to view Company 123's data.

Image /company/123/edit Only users who can edit Company 123 can access these pages.

Image /company/123/delete Deleting a company is restricted to superusers and the original creator.

A deny-by-default design automatically denies access to a request when a policy for that request has not been set or configured. Consider using Java filters to ensure that all requests go through your access control enforcement layer. When a developer adds a new feature to your system, the enforcement layer should automatically deny access to everyone until rules for that new feature are added to your access control model. It is much easier to accomplish this design when your application's policy enforcement layer is a Java filter. This kind of deny-by-default design makes your access control model more robust and fewer access control checks will be missed.

Image

**TIP**

*A filter allows you to perform consistent authorization checking routines on all application pages. It is much easier to be consistent when a centralized filter class drives access control enforcement, as opposed to authorization checks scattered throughout your code.*

A filter will allow you to easily reject requests for a certain URL path, or from certain IP addresses, or prevent anyone who is not on the corporate intranet from accessing admin functionality. This defense can be easily modified by editing config files and would not require code changes if designed correctly. In addition, it will make auditing your server access logs for security events much easier.

## **Positive Pattern 2: Build a Centralized Access Control Engine/Decision Maker**

While the policy enforcement layer applies the access control checks, the Policy Decision Point does the actual work to verify if the *subject* has the right access to conduct the *action* being requested. Again, this should be a centralized service as opposed to checks scattered throughout your code base.

## **Positive Pattern 3: Server-Side Trusted Data Should Drive Access Control Decisions**

Very little data needed for an access control decision should come from the request. The user/subject identity information should be server-side in a trusted session. The subject's metadata should all be retrieved server-side. The policy rules and other Policy Information Points should come from a trusted server-side resource. The only data elements that should come from the request (and can therefore be tampered with) should be the object's identity and the feature being requested. Retrieving role data from the request is bad practice because an attacker could easily make himself an admin or other privileged role!

## **Positive Pattern 4: Group Subjects, Objects, Actions, and Other Metadata**

In order to make administration more manageable, grouping is fundamental to good access control. At the very least, subjects and users can be grouped so policy rules need only be applied to the group instead of each individual member separately. For some kinds of access control design, actions and features that can be administered together are also good candidates for grouping. For example, a car management program may have features such as viewCar, editCar, diagnoseCar, and updateCar. All of these features can be rolled into one "feature group" of "carManagement" to make it easier to apply several features to one subject or subject-group. Grouping also makes it easier to audit policy for your user base because you don't have to worry about whether individual users have a certain permission—you will know based on the group or groups they are assigned to.

## **Positive Pattern 5: Be Able to Change a User's Entitlement in Real Time**

The moment your system changes the access control policy rules for a specific user (i.e., the user's entitlements) those changes should take place immediately. We have encountered numerous systems where the access control policy is loaded for a subject when he or she logs on, and not refreshed until the user logs off and back on again. It is easy for an attacker who has captured a valid user's session to set up a script to ping a site every few minutes, thus keeping that session, and thus that level of access, alive indefinitely.

It is critical for high-security systems to make access control policy changes take effect immediately. As a compromise between performance and security, we have built several systems where the access control policy is cached at login time and is refreshed every few minutes, based on performance and security needs. In such systems, it is also prudent to clear the loaded access control policy from the cache as soon as a change is made. This will force the policy to be reloaded on the next user request.

## **Positive Pattern 6: Access Control Policy Expressiveness**

An access control policy needs to be able to model the required functionality desired by the application. An access control mechanism with a great deal of expressiveness can model a wide variety of access control policies and handle input from a wide variety of sources when implementing those policies.



## Role-Based Access Control

Image

NOTE

The Unified NIST RBAC model was published in 2000<sup>1</sup> and was made the INCITS (InterNational Committee for Information Technology Standards) standard for role-based access control in 2004 (INCITS 359-2004).

In the world of web application authorization, role-based access control, or RBAC, is the most common design pattern you will see in Java frameworks. Many developers are familiar with adding role-based access control rules in individual pages and features within their Java application. The code likely looked something like this:

Image

Here is an example of a Spring access control rule:<sup>2</sup>

Image

The following is an Apache Shiro role assertion example:<sup>3</sup>

Image

The following example shows how a Struts 2 application can call Java servlet API access control checks utilizing JAAS. These are simple to add into code anywhere.

Image

The following is the Struts 2 RoleInterceptor core class at work for server-side role-based access control verification.<sup>4</sup> The Struts RoleInterceptor class protects individual actions to ensure the requesting user has the right role.

Image

Behind the scenes of any role-based access control system is some sort of persistence layer to store this policy data for each user. The preceding examples are all XML configuration or some kind of programmatic RBAC representation. It is also necessary to store the roles mapped to each user. Figure 3-3 is a sample database schema that could be used to model basic user, grouping, and role relationships as demonstrated earlier.

Image

---

**FIGURE 3-3.** Database schema modeling user, grouping, and role relationships

Optionally, the `subject_role_xref` table can be added to allow users to be directly associated with roles, without the need for a group. However, this will increase the complexity of the checks required at the Policy Decision Point.

Mapping roles to users is an easy model to grasp and set up. This makes the administration of RBAC policy simple to set up and maintain. RBAC also allows easy configuration of enforcement points in most frameworks. For basic hierarchical access control,

RBAC does the job quite well. Despite this ease of use, RBAC struggles when an application's authorization needs become more complex.

## RBAC Struggles: Data-Specific/Contextual Access Control

Another problem with RBAC is that modern implementations do not address *data-specific* or *context-specific* access control requirements and rules. For example, you may have several users with administrator access, but what if you wanted to limit each one to only specific organizations or specific rows in your database? Or a blog application where an author should have the edit rights to his own posts, but not to other people's. RBAC fails to address these core data-specific requirements that come up frequently in modern web application development. In order to merge data contextual requirements with access control, you often wind up with access control code that looks like this:

Image

Any time you want to change this policy, the code needs to change again! For example, what if you wanted to limit that feature with time constraints? Or allow other roles to execute this feature? What if you needed to perform this check in several areas of your application? You quickly end up with an access control mechanism that is not only confusing to understand (also known as spaghetti code) but also difficult to maintain. Role-based access control is appropriate for applications with modest access control functionality needs, but it does have its limits.

Image

**NOTE**

Chris Schmidt has an excellent and concise article on the limits of role-based access control that dates back to 2009.<sup>5</sup> He is not the only developer or analyst to discuss the limits of role-based access control as implemented in modern software frameworks.<sup>6</sup> Chris makes a good point here. If you think about it, the MVC pattern applied to software apps was revolutionary at the time. That's what needs to happen with application security models. Today you have the security model wedged into the application and it obscures your code. You need to decouple the security model and implementation from your applications. There's a lot of room for improvement.

## Multitenancy and Access Control

Multitenant web applications allow specialized groups of users to have specialized features of the application or private instances of the application. For example, suppose you had a web-based calendar application that allowed a company to register for the use of your sauna. Although all companies may be competing for the same time slots, groups of users from one company would only be able to see reservation details from their own company. A user with full access to one company's reservation may have only view access for another company, and only be able to see if a time slot is booked or free for others. The simple question of "What role is this user?" is no longer enough in the face of multitenancy. The question instead becomes "What role does this user play in company 4315?" or "What capabilities does this user have on article 12323?" And even these may not be enough to answer even more complex access control questions that are time, geography, or otherwise even more context dependent.

## Contextual Access Control

The web applications that we as developers are being tasked to build are only getting more complex over time. With that increased complexity, better access control design is required. So when you look at a simple two-role 1990s web application in comparison to a modern multitenant portal-based web application, the question of “What is access control?” changes significantly.

In today’s complex applications, authorization decisions need access to multiple data sources. Authorization decisions also need data from the environment (geolocation, time, and so on), they need additional information about the subject, they need information about the functionality the user wants to execute, they need information about the data the user is accessing or updating, and they need the actual policy rule or rules for the current request. There is actually a lot of *context* needed in order to make a complex access control decision in modern web software.

### Permission-Based Access Control and Apache Shiro

Permission-based access control is a term defined from the Apache Shiro project that addresses how programmers add access control checks (Policy Enforcement Points) in application code. “Permission” is admittedly an overloaded term and can stand for many things. In this context “permission” means what the Policy Enforcement Point looks like in code.

Permission-based access control involves defining action-based access control checks in application code (as opposed to checks that verify policy, like a role check). When this pattern is used in your application, you no longer have to modify code when your access control policy changes, regardless of how the back-end access control check is implemented!

When your Policy Enforcement Points are permission-based, the back-end access control decisions, the Policy Decision Points, can be implemented with RBAC, ABAC, or something different. Permission-based access control addresses only how enforcement points are coded in applications, which is a very important concept when it comes to managing the access control policy of a complex application.

Apache Shiro (<https://shiro.apache.org/authorization.html>) stated that, “a better way of performing access control is through permission-based authorization. Permission-based authorization, because it is strongly associated with your application’s raw functionality (and the behavior on an application’s core resources), permission-based authorization source code changes when your functionality changes, not when there is a security policy change. This means code is impacted much-less frequently than similar role-based authorization code.”

Let’s take a look at a few examples of enforcement points defined by the Apache Shiro project. Apache Shiro allows for permission-based enforcement points to verify if a user can execute a certain feature or activity. The following code example from Apache Shiro demonstrates the power of using permission-based access control patterns.

Image

Also, Apache Shiro can perform *instance-level* contextual permission checks. This not only verifies if a user can execute a certain feature, but can also check if the user has access to a specific piece of data.

Image

[Figure 3-4](#) is an example of a database schema that could model a basic form of object, permission, and entitlement relationships to users and roles.

**FIGURE 3-4.** *Database schema modeling object, permission, and entitlement relationships*

This database schema builds upon our earlier RBAC schema (keyed to the `subject_group` table) and adds modeling for contextual access control questions. These additions include:

Image How to model relationships between entities

Image How to store attributes for each entity

Image How to handle grouping

Image

**NOTE**

NIST released special publication (SP) 800-162, *A Guide to Attribute Based Access Control (ABAC)*. This standard addresses complex access control requirements where identity, subject and object attributes, environmental conditions, and an access control policy are all considered. Access control mechanisms of this nature are required for the next generation of enterprise and government applications (<http://nvlpubs.nist.gov/nistpubs/specialpublications/NIST.sp.800-162.pdf>).

## Spring Security 3.0 ACLs

Spring is one of the most popular frameworks and does indeed provide role-based access control out of the box. But how do you do data contextual or other more complicated access control checks in Spring 3+? The key is to build a custom access control Spring ACL (access control list) mechanism specific to each domain object instance. Each ACL, for each object instance, provides user (subject) rules around who can and cannot operate on that domain object.

Spring ACL Security 3.2 claims to satisfy almost every potential access control design need. It provides out-of-the-box RBAC. It provides a way of mapping access control rules specific to each object instance in your system. It allows for access control checks that are URL based, method based, and more. While feature complete, Spring Security is complex and requires heavy configuration and understanding of the platform. For more information, visit <http://docs.spring.io/spring-security/site/docs/3.2.x/reference/htmlsingle/#authorization>.

## ABAC Attribute-Based Access Control

ABAC, or attribute-based access control, is a formal, highly flexible access control framework that allows access control decisions to be based on attributes of users (subjects), objects, environmental conditions, and access control rules.

The most widespread use of ABAC is the *eXtensible Access Control Markup Language* (XACML). XACML can handle modeling of not only ABAC design, but also RBAC, which is essentially just a subset of ABAC anyhow.

Gunnar Peterson states, “It seems to me that the value of XACML and ABAC is really in the use cases that they enable. It’s outward focused, and unlocks value through new kinds of service.”<sup>7</sup>

Following are the discrete steps in ABAC access control processing:

1. The Policy Enforcement Point (PEP) receives an access request from some user (subject).

2. The PEP ensures that the user is authenticated.

3. The PEP constructs the XACML request, including the action being requested, the object the user is operating on, and the environmental values if applicable.

4. The PEP sends the request to the Policy Decision Point (PDP).

5. The PDP receives the request and necessary information, looks up the appropriate access control rules, and makes the access control decision.

6. Depending on the access control result, the PDP service constructs a XACML context response and returns it to the PEP with status, and obligations if any.

7. The PEP takes actions according to the response.

XACML is an incredibly complex but comprehensive and feature-rich access control policy modeling standard. Implementing ABAC is beyond the scope of this book and is only needed for the most complex access control requirements. For more information, we encourage you to read the NIST publications on the subject at <http://csrc.nist.gov/projects/abac/>.

## RBAC vs. ABAC

RBAC is all about roles while ABAC is all about attributes. In RBAC, roles also represent access control policy. In ABAC, roles are just attributes of users and contain all of the benefits of RBAC as a subset of ABAC's benefits.

The following table shows a few of the trade-offs between pure RBAC and ABAC implementations.

Image

---

### Access Control Cheatsheet

1. Avoid assigning permissions on a per-user basis. Group activities and users.

2. Perform consistent authorization checking routines on all application pages. A J2EE filter is ideal for access control.

3. Implement role-based access control for single-company, simple web applications.

4. Consider creating your enforcement points with permission-based calls when multitenant access control design or data contextual access control design is needed.

5. Consider attribute-based access control and XACML for the most complex and contextual access control needs.

6. Log all failed access authorization requests to a secure location for review by administrators. This provides awesome intrusion detection, as we discuss in [Chapter 9](#).

---

## Summary

In summary, role-based access control is an easy-to-understand access control pattern for simple, single-company web applications. Either permission-based access control (Apache Shiro) or Spring's ACL mechanism seems a better choice for web applications that require multitenancy or data-specific access control checks, but some customization will be needed for your application or environment. And finally, more advanced enterprise web applications that require access control policy based on environmental conditions, user data, object data, and other factors may require attribute-based access control, XACML, and the emerging tools in this space, but the complexity to build such access control is steep.

<sup>1</sup> <http://csrc.nist.gov/rbac/sandhu-ferraiolo-kuhn-00.pdf>

<sup>2</sup> <http://docs.spring.io/spring-security/site/docs/3.0.x/reference/el-access.html>

<sup>3</sup> <http://shiro.apache.org/authorization.html>

<sup>4</sup> <http://struts.apache.org/development/2.x/struts2-core/apidocs/org/apache/struts2/interceptor/RolesInterceptor.html>

<sup>5</sup> <http://yet-another-dev.blogspot.fr/2009/11/is-role-based-access-control-dead.html>

<sup>6</sup> [www.youtube.com/watch?v=MoLPsPPQS\\_Q](http://www.youtube.com/watch?v=MoLPsPPQS_Q)

<sup>7</sup> [http://1raindrop.typepad.com/1\\_raindrop/2013/06/security-140-conversation-with-gerry-gebel-on-xacml-and-abac.html](http://1raindrop.typepad.com/1_raindrop/2013/06/security-140-conversation-with-gerry-gebel-on-xacml-and-abac.html)